CyberGuard — Project Report

1. Project Title

CyberGuard: Python-Based Defensive Cybersecurity Toolkit

2. Abstract

CyberGuard is an educational cybersecurity application built with Python and Streamlit. It brings several defensive checks into one local dashboard: password hygiene analysis, file integrity verification, authentication-log analysis, and a limited TCP port check for authorized devices. The project demonstrates basic security concepts in a user-friendly interface.

3. Problem Statement

Beginners often use separate tools to understand password hygiene, verify file changes, and review authentication logs. CyberGuard provides a small, unified learning environment for these defensive tasks.

4. Objectives

• Build a simple browser-based security dashboard.

• Demonstrate SHA-256 hashing for file integrity.

• Identify repeated failed-login patterns in text logs.

• Provide password hygiene feedback without storing the entered password.

• Demonstrate basic TCP connectivity checks on explicitly authorized systems.

5. Technologies Used

• Python 3.10 or later

• Streamlit for the user interface

• Pandas for displaying tabular results

• hashlib for SHA-256 file hashing

• socket for TCP connection checks

• Regular expressions and collections for log parsing

• JSON for local integrity-baseline storage

6. Modules

6.1 Password Analyzer

Checks password length, letter case, digits, symbols, common-password matches, and repeated-character runs. The score is a simple educational heuristic, not a formal entropy or cracking-time estimate.

6.2 File Integrity Monitor

Calculates a file's SHA-256 digest and compares it with a saved local baseline. A digest mismatch indicates that the file content changed, though the tool cannot establish whether the change was malicious.

6.3 Security Log Analyzer

Looks for common authentication failure and success phrases, counts events, and flags IP addresses that exceed a user-selected failed-login threshold. Results depend on the input log format.

6.4 Authorized TCP Port Check

Attempts TCP connections to a limited list of user-selected ports. It is not a vulnerability scanner; firewalls and network conditions may affect results.

7. System Requirements

• Windows, macOS, or Linux

• Python 3.10+

• Internet connection for initial dependency installation

• Modern web browser

8. Installation and Execution

1. Extract the project folder.

2. Create and activate a Python virtual environment.

3. Install dependencies using `pip install -r requirements.txt`.

4. Launch the dashboard using `streamlit run app.py`.

5. Open the local URL shown in the terminal, normally `http://localhost:8501`.

9. Testing Plan

• Password Analyzer: test short, common, and longer sample passwords.

• File Integrity: save a baseline, change the file, and confirm that the digest differs.

• Log Analyzer: use `sample_data/auth.log` and verify that repeated failed attempts are flagged at the configured threshold.

• Port Check: check a few ports on `127.0.0.1`, which refers to the local machine.

10. Expected Outcome

The user can explore four defensive security concepts from one dashboard and view the results in readable summaries and tables.

11. Limitations

• Password scoring is heuristic and should not be treated as a security guarantee.

• Log parsing relies on recognizable text patterns.

• Baselines stored on the same machine can be altered by an attacker with access to that machine.

• Port status results can be affected by firewall rules and timeouts.

• The application does not detect all malware, block attacks, or replace a professional security solution.

12. Future Scope

Possible extensions include automated tests, exportable reports, folder watching, more log formats, stronger baseline protection, and authentication before public deployment.

13. Conclusion

CyberGuard is a practical introductory project for learning Python application development and defensive cybersecurity fundamentals. It is designed for local educational use and authorized security checks.

14. References

• Python documentation: https://docs.python.org/3/

• Streamlit documentation: https://docs.streamlit.io/

- Python `hashlib`: https://docs.python.org/3/library/hashlib.html
- Python `socket`: https://docs.python.org/3/library/socket.html